

WarpDrive: GPU-Based Fully Homomorphic Encryption Acceleration Leveraging Tensor and CUDA Cores

- 核心目标：高效数论变换
- 方法：算子融合、线程束特化同时利用CUDA Core和Tensor Core

CKKS协议 - 密码学计算 - 算子/硬件

CKKS中数论变换的计算特性

- 域元素位宽一般为64位以内，在GPU上可以通过双素数重缩放或者有理重缩放技巧控制在32位以内
- 阶数在 2^{20} 以内

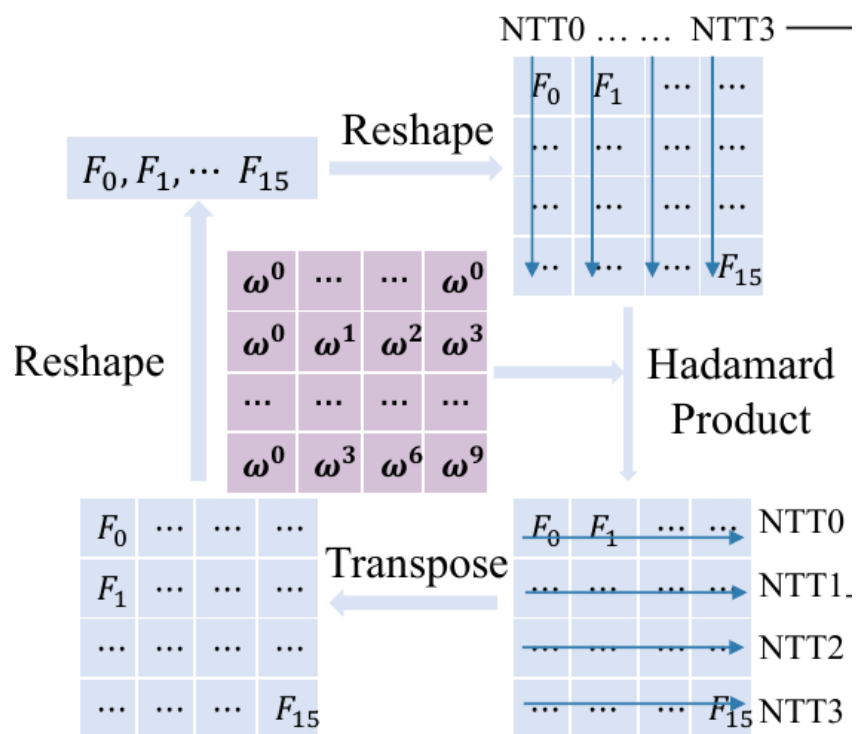
GPU上的整数算力(A100)

- CUDA Core: 19.5 TOPS INT32
- Tensor Core: 624 TOPS INT8
- $\frac{624}{16} = 39$

挑战

- 没法直接算子融合，因为 2^{20} 个8字节远超片上内存
- 解决方案：使用四步NTT拆分子问题

四步NTT



阶数为 2^n 的1个数论变换

→ 阶数为 $2^{\frac{n}{2}}$ 的 $2^{\frac{n}{2}}$ 个数论变换

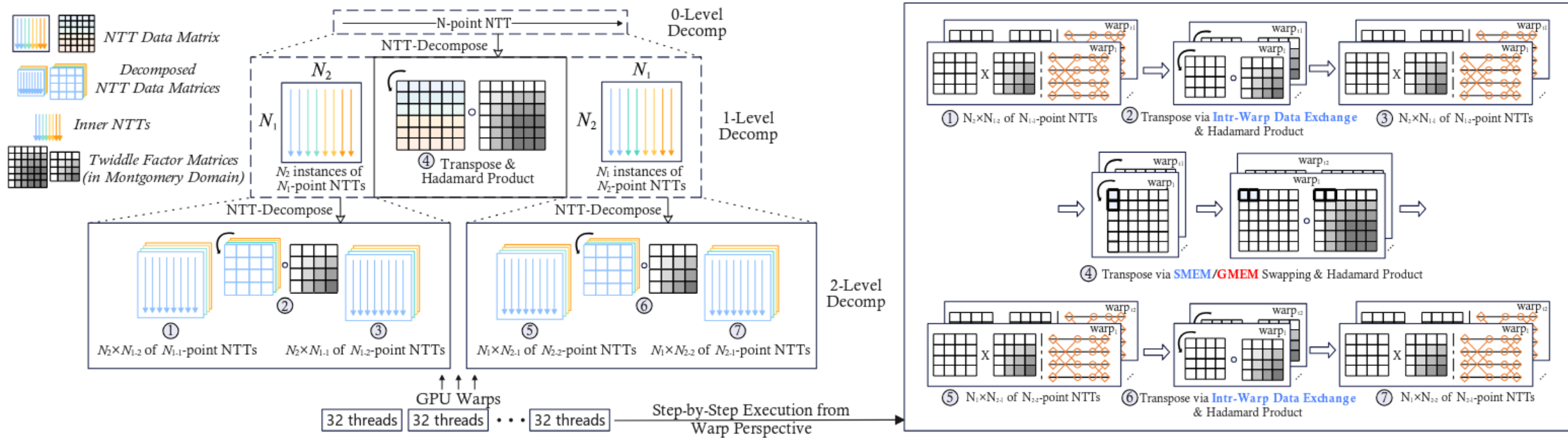
对于 $n = 20$,

→ 阶数为 2^{10} 的1024个数论变换

挑战

- 直接使用矩阵乘法计算 2^{10} 阶NTT需要的 ω 的幂 (twiddle系数) 太多了 (2^{20})，无法放在片上内存(TensorFHE)
- 解决方案：用四步NTT再拆一次
→ 阶数为 2^5 的32个NTT

WarpDrive



CUDA Core: 使用相比矩阵乘法更高效的Cooley Turkey算法 ($n \log n$ 复杂度) 计算NTT, 并且不需要拆分为8位整数

TABLE VII: Throughput Comparison of NTT and INTT Operations for CPU, TensorFHE and WarpDrive (KOPS).

Operation	Scheme	SET-A	SET-B	SET-C	SET-D	SET-E
NTT	CPU Baseline [49]	7.2	3.4	1.6	-	-
	TensorFHE [22] *	910	450	209	98.9	48.3
	WarpDrive	12181	4675	2088	1009	468
	Speedup Over CPU	1692 $\times$	1375 $\times$	1305 $\times$	-	-
	Speedup Over [22]	13.4$\times$	10.4$\times$	10.0$\times$	10.2$\times$	9.7$\times$
INTT	CPU Baseline [49]	7.6	3.5	1.7	-	-
	TensorFHE [22] *	913	449	209 W	98.8	48.1
	WarpDrive	12097	4643	2073	1009	465
	Speedup Over CPU	1592 $\times$	1327 $\times$	1223 $\times$	-	-
	Speedup Over [22]	13.2$\times$	9.9$\times$	10.0$\times$	10.1$\times$	9.7$\times$

*: TensorFHE performance data corresponding to SET-A, SET-B, and SET-C are from [22]. SET-D and SET-E are tested on NVIDIA-PCIE-A100 using its open-source NTT implementation [4].

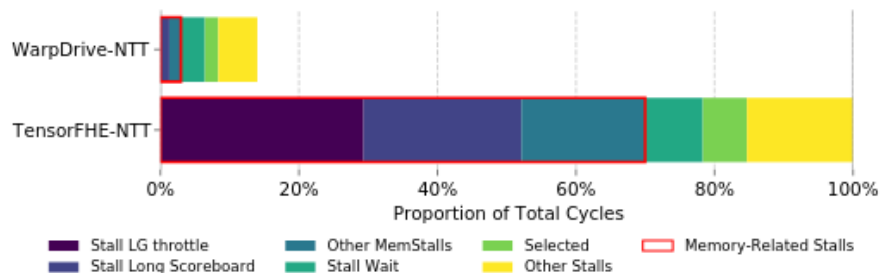


Fig. 5: Scheduler Cycles Breakdown in TensorFHE-NTT and WarpDrive-NTT (WD-Tensor Variant) ($N = 2^{16}$, $batch_size = 1024$).

TABLE VI: HE Parameter Sets Used in the Evaluation of NTT and Homomorphic Operations.

	Set-A	Set-B	Set-C	Set-D	Set-E
N	2^{12}	2^{13}	2^{14}	2^{15}	2^{16}
l	2	6	14	24	34
$\log qp$	108	217	437	704	974

反思

Algorithm 1 Kernel-Level Task Assignment for Tensor Core-based NTT

```
1: SplitKernel:  
2:   Load  $X$  from GMEM  
3:   Split UINT32  $x_i$  in  $X$  into 4 UINT8 elements in  $X_m$   
4:   Store  $X_m$  to GMEM  
5: GEMMKernel  
6:   Load  $X_m$  and  $W_{1n}/W_{3n}$  from GMEM  
7:   Perform GEMM of  $X_m$  and  $W_{1n}$  with Tensor Cores  
8:   Store  $Y_{mn}$  to GMEM  
9: MidKernel  
10:  Load  $Y_{mn}$  and  $W_2$  from GMEM  
11:  Reassembling 16 elements in  $Y_{mn}$   
12:  Perform ModRedc and hadamard production with  $W_2$   
13:  Split back into four UINT8 elements in  $X_m$   
14:  Store  $X_m$  to GMEM  
15: MergeKernel  
16:  Load  $Y_{mn}$  from GMEM  
17:  Reassembling 16 elements in  $Y_{mn}$  and perform ModRedc  
18:  Store  $X$  to GMEM  
19: HostFunction FIVE_STAGE_NTT  
20:   $(X_1, X_2, X_3, X_4) \leftarrow \text{SPILT\_KERNEL}(X)$   
21:  for  $m \leftarrow 1$  to 4;  $n \leftarrow 1$  to 4 do  
22:     $Y_{mn} \leftarrow \text{GEMM\_KERNEL}(X_m, W_{1n})$   
23:   $(X_1, X_2, X_3, X_4) \leftarrow \text{MID\_KERNEL}(\{Y_{mn} \mid 1 \leq m, n \leq 4\}, W_2)$   
24:  for  $m \leftarrow 1$  to 4;  $n \leftarrow 1$  to 4 do  
25:     $Y_{mn} \leftarrow \text{GEMM\_KERNEL}(X_m, W_{3n})$   
26:   $X \leftarrow \text{MERGE\_KERNEL}(\{Y_{mn} \mid 1 \leq m, n \leq 4\})$ 
```

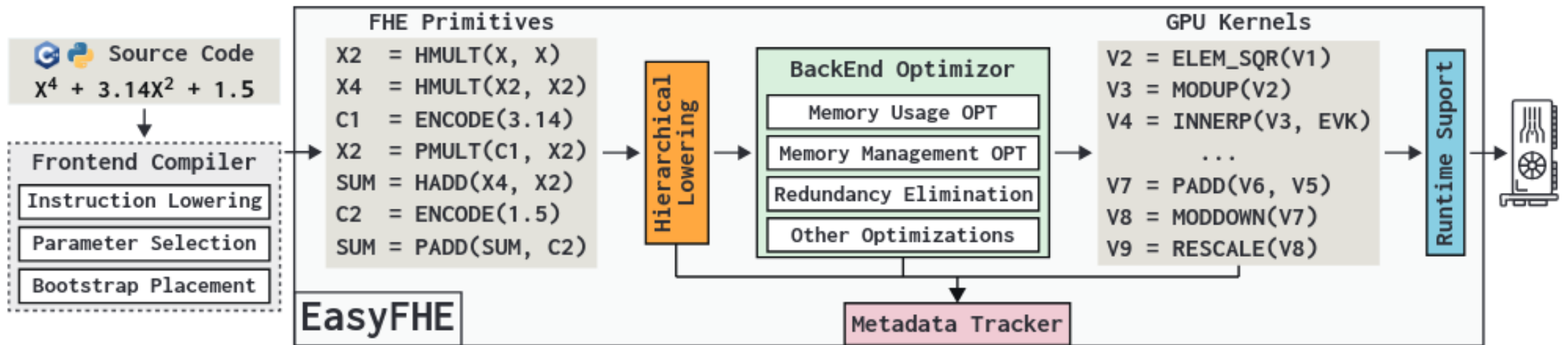
Algorithm 2 Warp-Level Task Assignment for Tensor Core-based NTT

```
1: OneStageNTTKernel:  
2:   Load  $X$  and  $W_{1n}/W_{3n}$  from GMEM  
3:   Split UINT32  $x_i$  in  $X$  into 4 UINT8 elements in  $X_m$   
4:   Store  $X_m$  and  $W_{1n}/W_{3n}$  to SMEM  
5:   Load  $X_m$  and  $W_{1n}/W_{3n}$  from SMEM  
6:   Perform GEMM of  $X_m$  and  $W_{1n}$  with Tensor Cores  
7:   Store  $Y_{mn}$  to SMEM  
8:   Load  $W_2$  from GMEM  
9:   Load  $Y_{mn}$  from SMEM  
10:  Reassembling 16 elements in  $Y_{mn}$   
11:  Perform ModRedc and Hadamard Production with  $W_2$   
12:  Split back into four UINT8 elements in  $X_m$   
13:  Store  $X_m$  to SMEM  
14:  Load  $X_m$  and  $W_{1n}/W_{3n}$  from SMEM  
15:  Perform GEMM of  $X_m$  and  $W_{1n}$  with Tensor Cores  
16:  Store  $Y_{mn}$  to SMEM  
17:  Load  $Y_{mn}$  from SMEM  
18:  Reassembling 16 elements in  $Y_{mn}$  and perform ModRedc  
19:  Store  $X$  to GMEM  
20: HostFunction ONE_STAGE_NTT  
21:  ONESTAGENTTKERNEL( $X$ )
```

the warp level, eliminating the need to allocate separate kernels for each GEMM computation. This can prevent the frequent on-chip/off-chip data exchanges thereby avoiding prolonged

- 预处理、后处理对CUDA Core的争用如何？
- 如何分配共享内存？

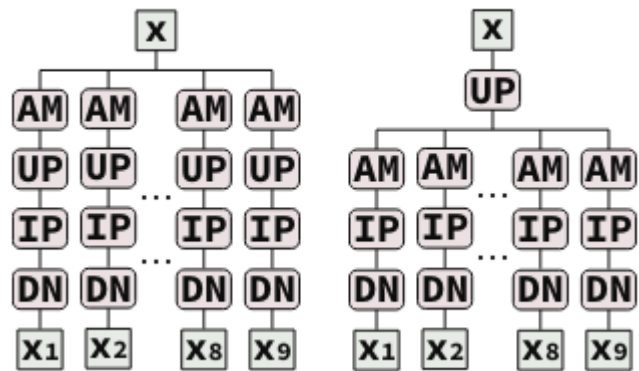
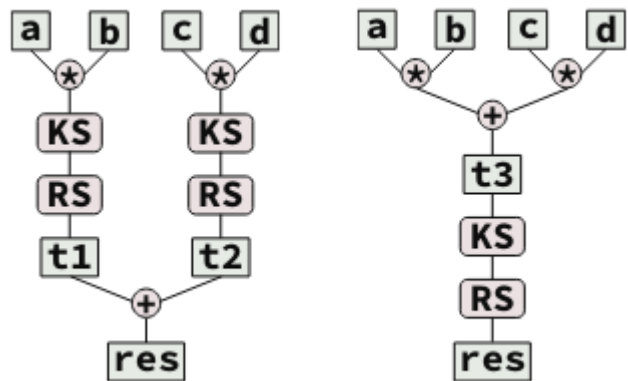
EasyFHE: A Framework for Developing and Optimizing Fully Homomorphic Encryption Programs on GPUs



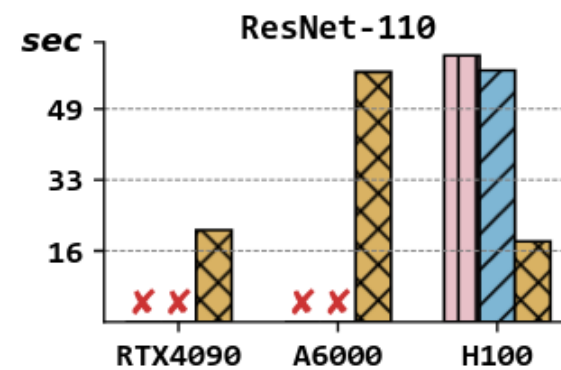
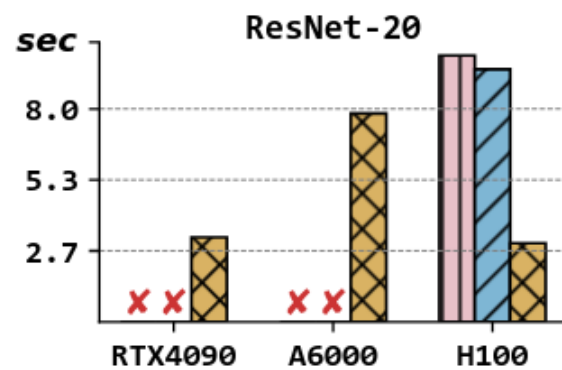
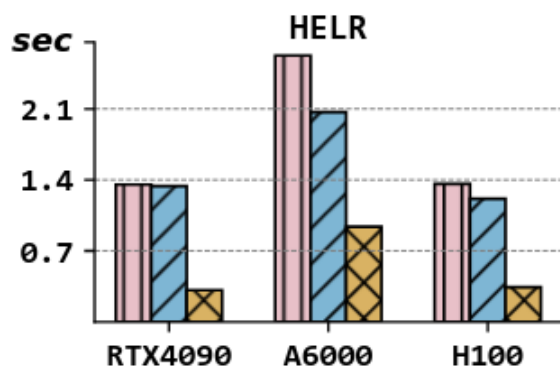
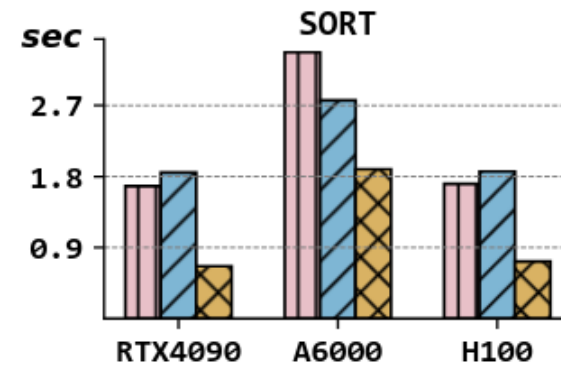
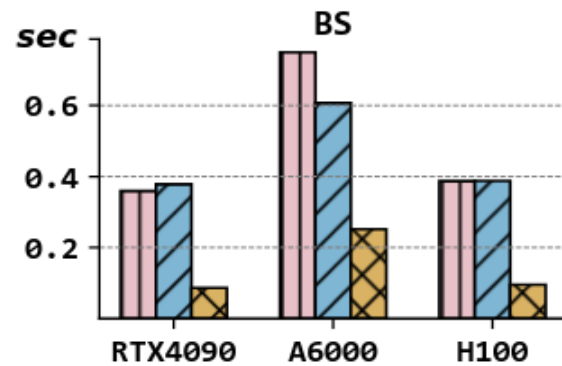
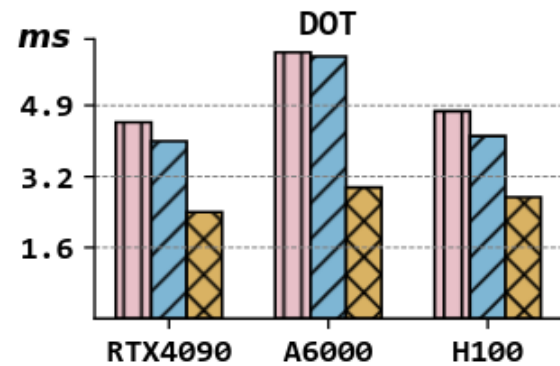
CKKS协议 - 密码学计算 - 算子/硬件

后端优化包括

- 预计算/计算平衡
- 公共子式消除
- 模式匹配优化
- 计算图调度



两种模式匹配优化



Troy

HEonGPU

EasyFHE

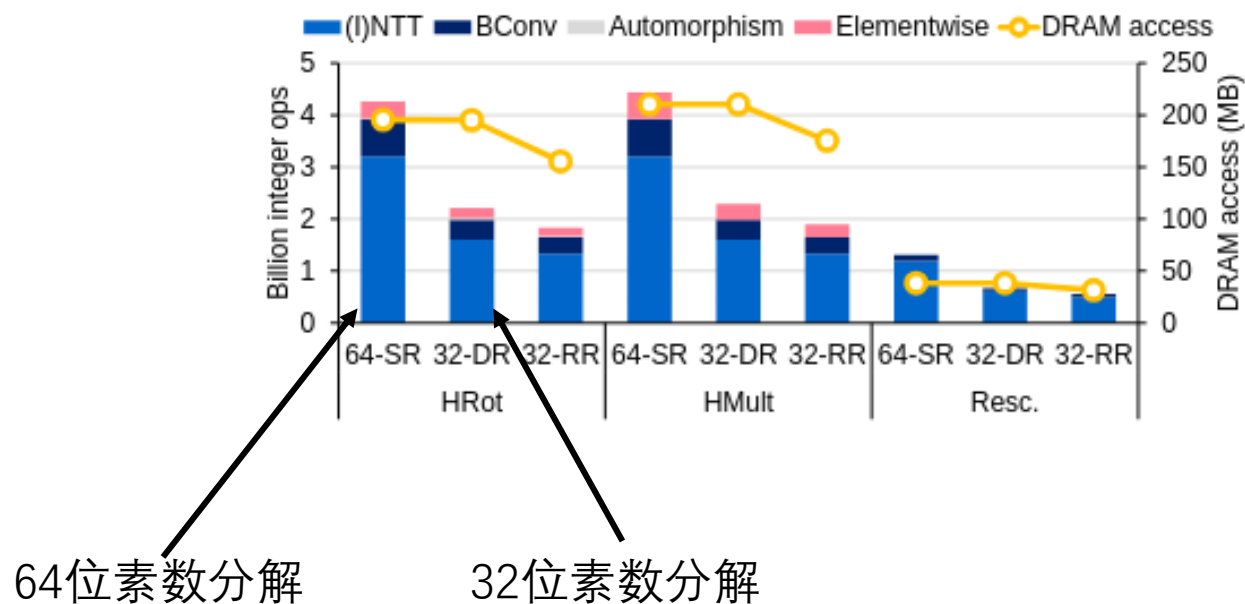
Cheddar: A Swift Fully Homomorphic Encryption Library Designed for GPU Architectures

- 低预计算开销的有理重缩放方法
- 一些算子融合

CKKS协议 - 密码学计算 - 算子/硬件

为什么要有理重缩放？

- GPU上64位整数运算是用32位运算组合的，因此RNS分解密文多项式到32位素数后总计算量大幅减小



为什么要有理重缩放

- 然而，缩放系数一般大于 2^{32} ，例如 2^{40} 。如果要对

$$Q_L = q_L Q_{L-1} = \cdots = q_L q_{L-1} \cdots q_1 Q_0$$

- 满足 $q_i < 2^{32}$ ，怎么每次重缩放 2^{40} ？

为什么要有理重缩放

$$Q_L = q_L Q_{L-1} = \cdots = q_L q_{L-1} \cdots q_1 Q_0$$

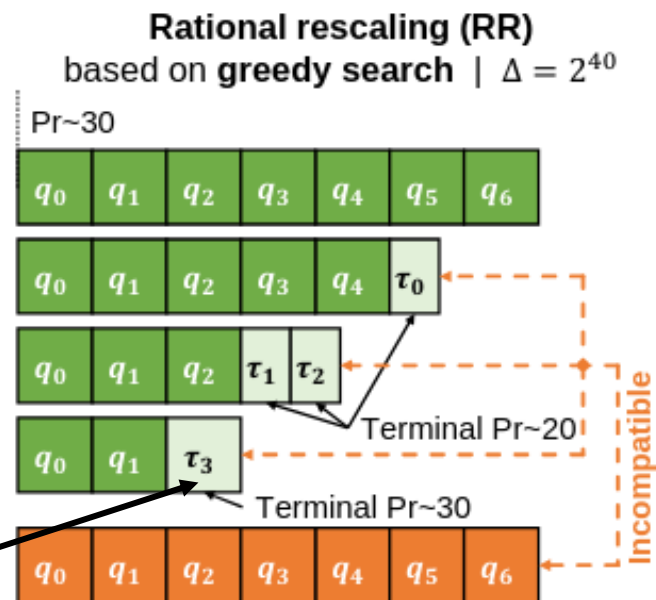
双素数重缩放：一次丢掉两个素数

- 问题：没有足够的素数满足NTT充要条件 $q_i = 1 \pmod{2N}$ ，从而限制了乘法深度

有理重缩放

怎么将密文和模数都除以 $\Delta \approx 2^{40}$?

- 先乘以一个接近 2^{20} 的素数，再除以两个接近 2^{30} 的素数



终端素数

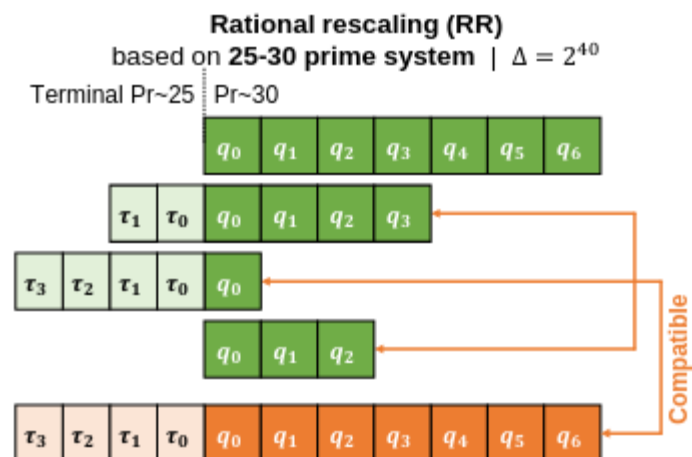
有理重缩放

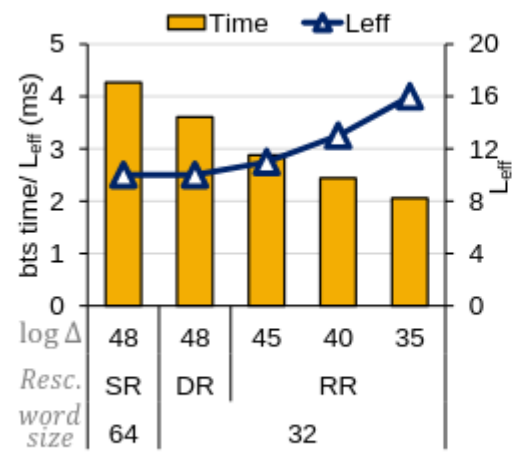
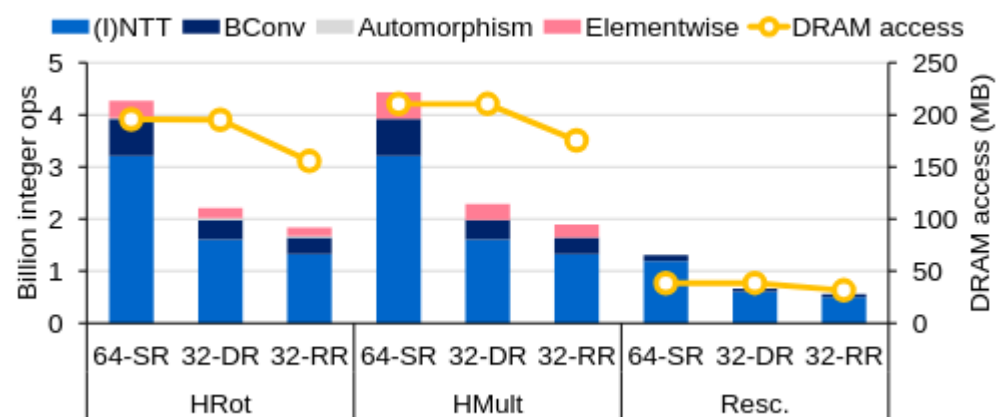
终端素数怎么找？

- 每层新找一个
 - 不好，终端素数太多导致每层都需要自己的EK
- 解决方案：为 $\Delta = 2^{30}, 2^{35}, 2^{40}, 2^{45}, 2^{50}$ 分别手工寻找可以重复使用的少数终端素数

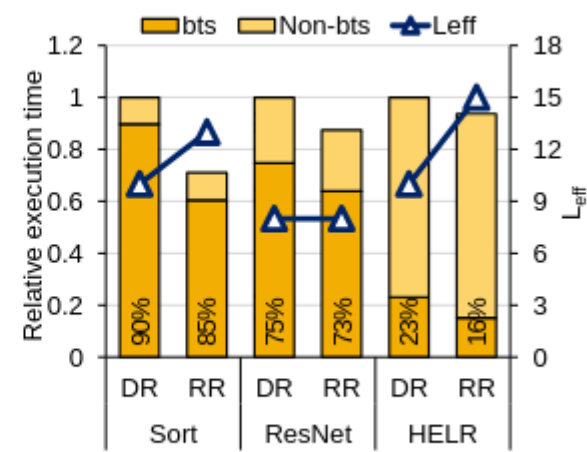
循环使用终端素数

$$\begin{aligned} -2 \times 25 + 3 \times 30 &= 40 \\ 4 \times 25 - 2 \times 30 &= 40 \end{aligned}$$





(a) (bts time) / L_{eff} & L_{eff}



(b) bts portion & L_{eff}